

Vector Cosine Similarity

How Quality AI Understands Meaning & Finds Past Solutions Without Matching Words

< 384-Dimensional Embeddings

Cosine Angle Math

Semantic Search Engine

RAG Knowledge Pipeline

all-MiniLM-L6-v2 AI Model

384

DIMENSIONS PER
EMBEDDING

0 → 1.0

SIMILARITY SCORE
RANGE

< 5ms

SEARCH LATENCY

100%

MEANING-BASED (NO
KEYWORDS)

1. What is a Vector? (The Foundation)

A **Vector** is a list of numbers that represents the **meaning and concept** of a sentence. Think of it as a GPS coordinate — but instead of 2 numbers (Latitude, Longitude), a sentence is mapped into **384 numbers** that capture its full meaning.

ANALOGY

Words as Map Coordinates

Just as the GPS coordinates **[12.9716, 77.5946]** uniquely identify Bengaluru on a map, the Quality AI model converts the sentence *"Paint peeling on car bumper"* into a 384-number coordinate like **[0.24, -0.81, 0.45, 0.12, 0.67, ...]** that uniquely identifies its *meaning* in concept-space.

🔑 The Golden Rule

Sentences with similar meaning will have vectors pointing in the same direction.

"Paint peeling on bumper" and "Coating detachment on vehicle panels" use completely different words — but an AI model trained on millions of engineering texts assigns both sentences vectors that point within 12–15° of each other in 384-dimensional space.

2. 2D Geometric Diagram — How Vectors Represent Meaning

Even though Quality AI uses 384 dimensions, we can visualize the core idea in 2D to understand angle and similarity:

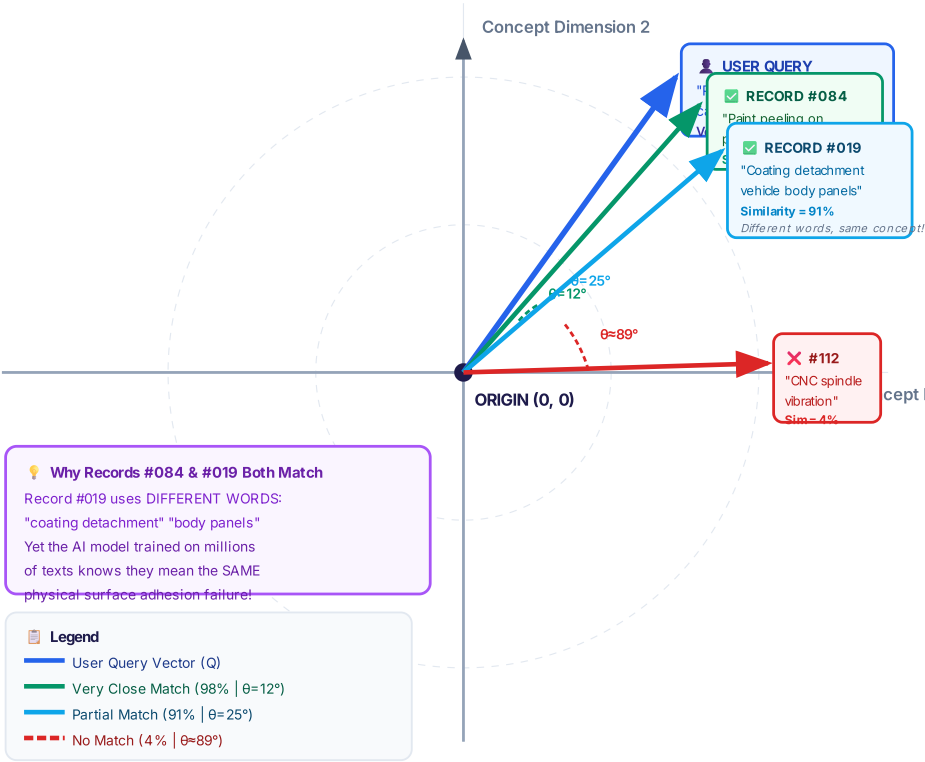


Figure 1: 2D representation of 384-dimensional vector space — arrows pointing in the same direction = similar meaning; arrows pointing sideways = unrelated topics

3. Live Example: "How was paint peeling solved before?" — Full Data Flow

1

User Types a Natural Language Question

The quality engineer types into the AI chat widget embedded in the QCMS dashboard.

```
"How was paint peeling on automotive bumpers solved before?"
```

2

Authentication & Tenant Lock (rag_routes.py)

The backend verifies the user's JWT token. All database lookups are locked strictly to the user's `org_id`, so Company A can never see Company B's data.

```
POST /api/rag/chat → JWT verified → org_id = 5 locked
```

3

Sentence → Numbers (Embedding Conversion)

The **all-MiniLM-L6-v2** AI model converts the full sentence into a 384-number vector. This encoding captures the semantic meaning of every word, the sentence structure, and the engineering domain context.

```
Query Vector Q = [0.241, -0.812, 0.455, 0.122, 0.671, -0.330, ... (381 more numbers)]
```

4

Fetch All Past Project Vectors (PostgreSQL)

The system queries the `knowledge_repository` table to fetch all previously ingested project records for this organization — along with their pre-stored 384-dimensional embedding vectors.

```
SELECT id, title, problem_summary, root_cause, solution_summary, embedding FROM  
knowledge_repository WHERE org_id = 5
```

5

Cosine Similarity Computation (The Mathematical Core)

For each past project vector **V**, the system computes the Cosine Similarity against the user's Query vector **Q** using the dot product formula. Results are ranked from highest to lowest score.

```
Similarity(Q, V) = (Q · V) / (||Q|| × ||V||) → sorted descending
```

6

Top 4 Matches Selected & Formatted

The system selects the top 4 records with similarity above threshold, extracts their problem summary, verified root cause, countermeasure plan, KPI improvement %, and cost savings (₹), then formats the response with clickable project storyboard links.

```
Return: { answer: "Here are the top 2 matching past solutions...", sources: [{ project_id: 84,  
similarity: 0.98 }, { project_id: 19, similarity: 0.91 }] }
```

7

Verified Answer Rendered in Chat UI

The frontend renders the answer with formatted Markdown — bold highlights, bullet points, KPI metric badges, and clickable direct hyperlinks to the original quality storyboard project for full root-cause drill-down.

```
🔗 → /project-details.html?id=84 | /project-details.html?id=19
```

4. The Math Made Simple — Numerical Walkthrough

For illustration, we simplify to **3 dimensions** (in reality it is 384). The same formula applies identically.

4.1 — The Three Vectors

Vector Name	Sentence It Represents	3D Coordinates
Q (User Query)	"Paint peeling on automotive bumper"	[1, 2, 3]
V₁ (Past Project #084)	"Paint peeling defect on Line 2 plastic bumpers"	[2, 4, 6] (same direction, scaled 2×)
V₂ (Past Project #112)	"CNC milling spindle bearing vibration failure"	[5, -2, 0] (different direction)

4.2 — The Cosine Similarity Formula

```
// The Formula: Similarity(Q, V) = ( Q · V ) / ( ||Q|| × ||V|| ) // Where: // Q · V = Dot Product (how much two vectors "agree" in direction) // ||Q|| = Magnitude (length) of Query vector // ||V|| = Magnitude (length) of Past project vector
```

4.3 — Calculation: Query Q vs. Past Project V₁ (Paint Peeling Project #084)

Step A

Dot Product: $Q \cdot V_1$

$$\begin{aligned} Q \cdot V_1 &= (1 \times 2) + (2 \times 4) + (3 \times 6) \\ &= 2 + 8 + 18 \\ &= 28 \end{aligned}$$

Step B

Magnitudes: $||Q||$ and $||V_1||$

$$\begin{aligned} ||Q|| &= \sqrt{1^2 + 2^2 + 3^2} = \sqrt{1 + 4 + 9} = \sqrt{14} = 3.742 \\ ||V_1|| &= \sqrt{2^2 + 4^2 + 6^2} = \sqrt{4 + 16 + 36} = \sqrt{56} = 7.483 \end{aligned}$$

Step C

Final Cosine Similarity Score

$$\begin{aligned} \text{Similarity}(Q, V_1) &= 28 / (3.742 \times 7.483) \\ &= 28 / 27.99 \\ &= 1.000 \rightarrow 100\% \text{ EXACT MATCH! } \checkmark \end{aligned}$$

4.4 — Calculation: Query Q vs. Unrelated Project V₂ (CNC Bearing #112)

Step A Dot Product: $Q \cdot V_2$

$$\begin{aligned} Q \cdot V_2 &= (1 \times 5) + (2 \times -2) + (3 \times 0) \\ &= 5 + (-4) + 0 \\ &= 1 \end{aligned}$$

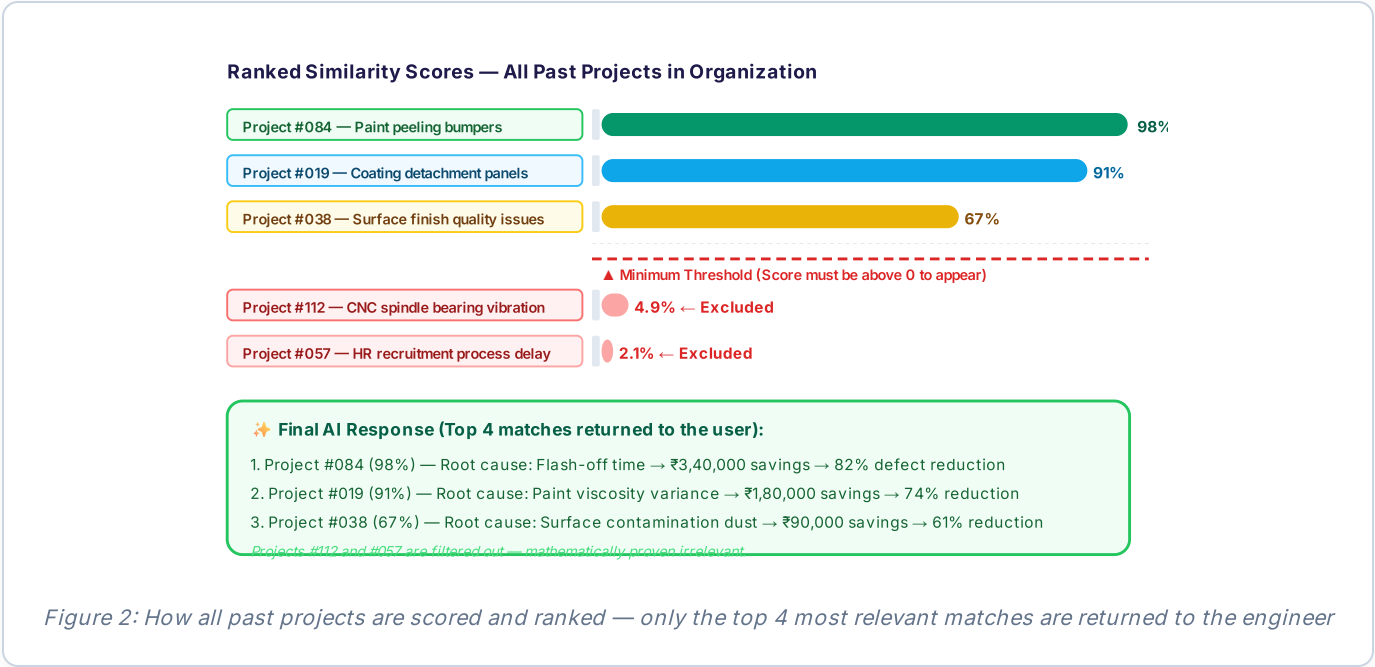
Step B Magnitude of V_2

$$||V_2|| = \sqrt{5^2 + (-2)^2 + 0^2} = \sqrt{25 + 4 + 0} = \sqrt{29} = 5.385$$

Step C Final Cosine Similarity Score

$$\begin{aligned} \text{Similarity}(Q, V_2) &= 1 / (3.742 \times 5.385) \\ &= 1 / 20.15 \\ &= 0.049 \rightarrow 4.9\% \text{ REJECTED as Irrelevant! } \times \end{aligned}$$

5. How Results are Ranked & Filtered



6. Why Vector Cosine Similarity vs. Traditional Search

Feature	Traditional Keyword Search ❌	Quality AI Vector Search ✅
How It Searches	Looks for exact character-by-character word matches in text.	Converts sentences into 384-number mathematical meaning-vectors and computes angular distance.
Synonym Handling	❌ "Coating detachment" will NOT match "paint peeling".	✅ "Coating detachment" and "paint peeling" are placed within 12° of each other in vector space → matched automatically.
Result Ranking	Lists all documents containing the keyword — no intelligence in ranking.	Every document receives a precise 0.0–1.0 similarity score → mathematically sorted most-relevant first.
Context Awareness	Ignores sentence structure and engineering domain context entirely.	Understands that "peeling on automotive production line" is completely different from "peeling a banana."
Multi-Language Concepts	Fails completely if a term is expressed differently.	Understands industry synonyms: Delamination = Peeling = Detachment = Adhesion failure.

 **Core Principle**
Cosine Similarity does not compare words — it compares the meaning-direction of vectors.

When the angle (θ) between two vectors is small, both sentences are conceptually similar — regardless of which exact words were used to express that concept. This is why Quality AI can retrieve past solutions even when engineers describe the same defect using completely different technical vocabulary.